Pokémon Wordle:
Intelligent Guessing Strategies

An Interactive Game and Algorithmic Strategy Comparison

Academic Year 2026–27

Team Members

Badagoni Sri Sahasra (B24DS003)

Malraju Srinayana (B24CS024)

Nagineni Thanmaie (B24EC019)

Parinitha Pulipati (B24CS037)

V. Varshitha (B24ME050)

1. Project Overview

Pokémon Wordle is an interactive guessing game inspired by the Wordle concept. The objective is to identify a hidden Pokémon from a dataset within a limited number of guesses.

The system combines an interactive gameplay environment with algorithms that attempt to identify the hidden Pokémon efficiently. Four guessing strategies are implemented:

- **Random:** selects an unguessed Pokémon at random, without using feedback.
- **Filter:** eliminates candidates that are inconsistent with the feedback received so far.
- **Entropy:** selects the guess with the greatest expected information gain.
- **Minimax:** selects the guess that minimizes the size of the largest remaining candidate group.

The application is built with Streamlit and offers three main modes of interaction: Human Gameplay, AI Demonstration, and Algorithm Comparison. Together they allow users to play manually, to observe automated decision-making step by step, and to compare the performance of the strategies using benchmark results. The application also includes a home page and a dataset explorer for browsing the Pokémon records.

2. Dataset and Game Design

2.1 Dataset Description

The game uses a dataset of 386 Pokémon records. Each record contains six attributes that are used for identification and comparison, as listed in Table 1.

Table 1: Pokémon dataset attributes

Attribute	Description
Name	Pokémon name
Generation	Generation to which the Pokémon belongs
Type 1	Primary Pokémon type
Type 2	Secondary Pokémon type, if available
Height	Pokémon height
Weight	Pokémon weight

These attributes provide the information needed to compare a guess with the hidden Pokémon and to generate feedback.

2.2 Game Rules

The game follows a fixed-guess structure:

- One Pokémon is selected at random as the hidden answer.
- The player or the AI submits a Pokémon as a guess.
- The system compares the guess with the hidden Pokémon and returns feedback.

- The feedback is used to choose the next guess.
- A maximum of eight guesses is allowed per game.
- The game ends when the correct Pokémon is identified or all eight guesses have been used.

Each guess returns one clue for each of five attributes: generation, primary type, secondary type, height, and weight. A clue indicates a match, a mismatch, or, for numerical attributes, whether the hidden value is higher or lower. Height and weight comparisons use a 10% tolerance band.

3. Guessing Strategies

Four algorithms are implemented to investigate how different guess-selection approaches affect game performance.

3.1 Random Strategy

The Random strategy selects a Pokémon at random from those that have not yet been guessed. It does not use feedback from previous guesses. It serves as the baseline against which the intelligent strategies are evaluated.

3.2 Filter Strategy

The Filter strategy maintains the set of Pokémon that are still possible. After each guess, it uses the feedback to eliminate candidates that do not satisfy the observed constraints. The next guess is chosen at random from the remaining candidates, so the search space shrinks progressively.

3.3 Entropy Strategy

The Entropy strategy selects guesses based on expected information gain. For a candidate guess, the possible feedback outcomes divide the remaining Pokémon into groups. The strategy measures the uncertainty of this division and prefers guesses that are expected to reveal the most information. The entropy of a probability distribution is

$$H(X) = -\sum_{i} p_i \log_2(p_i), \tag{1}$$

where p_i is the probability of the i-th feedback outcome.

3.4 Minimax Strategy

The Minimax strategy evaluates the feedback partitions that each possible guess would produce and selects the guess that minimizes the size of the largest remaining candidate group:

$$g^* = \arg\min_{g} \max_{r} |S_{g,r}|, \tag{2}$$

where

- g is a possible guess,
- r is a possible feedback outcome, and

- $S_{g,r}$ is the set of candidates consistent with outcome r after guess g.

This strategy focuses on limiting the worst-case search space rather than maximizing average information gain.

4. System Architecture and Implementation

The project follows a modular architecture in which each responsibility is handled by a separate Python module:

- **Dataset loading:** loads and prepares the Pokémon records.
- **Game mechanics:** manages the hidden answer, guesses, feedback generation, and game termination.
- **Solver modules:** implement the Random, Filter, Entropy, and Minimax strategies.
- **Benchmarking:** runs automated games and records performance measurements.
- **Streamlit interface:** provides the interactive application and visualizes game progress and results.

This separation of game logic, decision-making, benchmarking, and user interaction makes the application easier to maintain and extend.

5. Application Interface

The application is developed with Streamlit and deployed as a web application. It uses a retro pixel-art visual style, and its three main modes are described below. The screenshots in this section were captured from the deployed application.

5.1 Human Gameplay

In Human Gameplay, the user chooses Pokémon guesses manually. After each guess the application displays five clue tiles (generation, both types, height, and weight) and tracks the remaining attempts. Figure 1 shows a completed game that was solved in three guesses.

5.2 AI Demonstration

In the AI Demonstration mode, the user selects a secret Pokémon and one of the four strategies. The selected algorithm then plays the game, and its decisions are revealed one turn at a time. Each turn shows the guess, the clues received, and the number of candidates that remain. Figure 2 shows the first turn of the Entropy strategy, which reduces the candidates from 386 to 24.

5.3 Algorithm Comparison

The Algorithm Comparison mode presents the benchmark results. A scoreboard ranks the strategies by average guesses, and bar charts compare their performance. Figure 3 shows the scoreboard and the average-guesses chart.

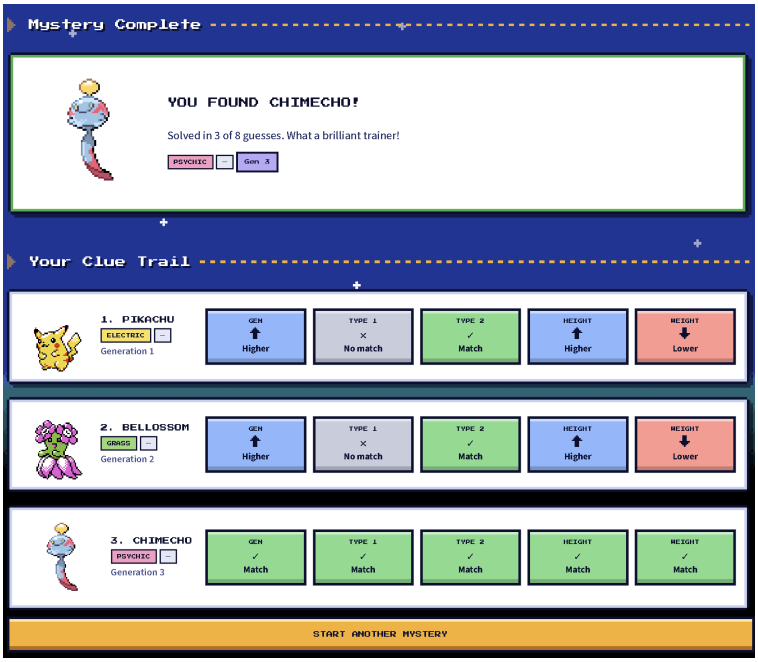


Figure 1: Human Gameplay: a completed game with the clue trail.

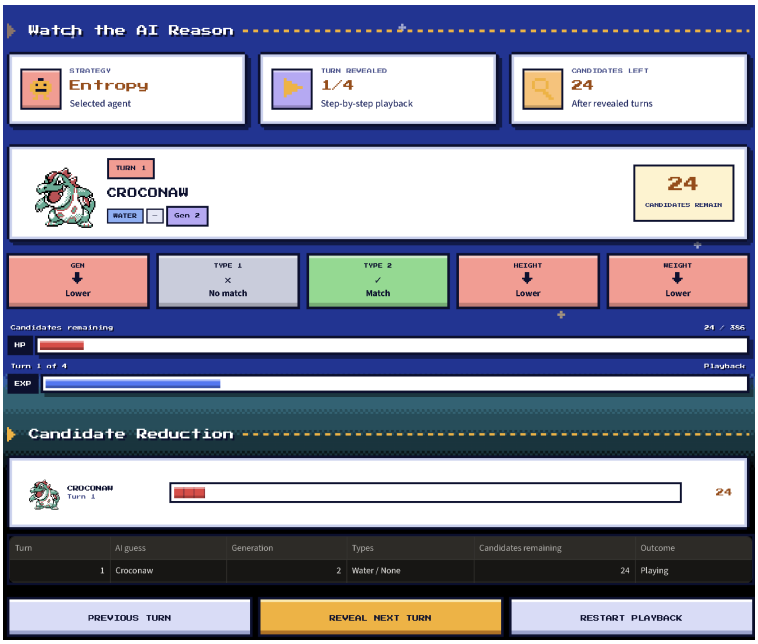


Figure 2: AI Demonstration: the first turn of the Entropy strategy.

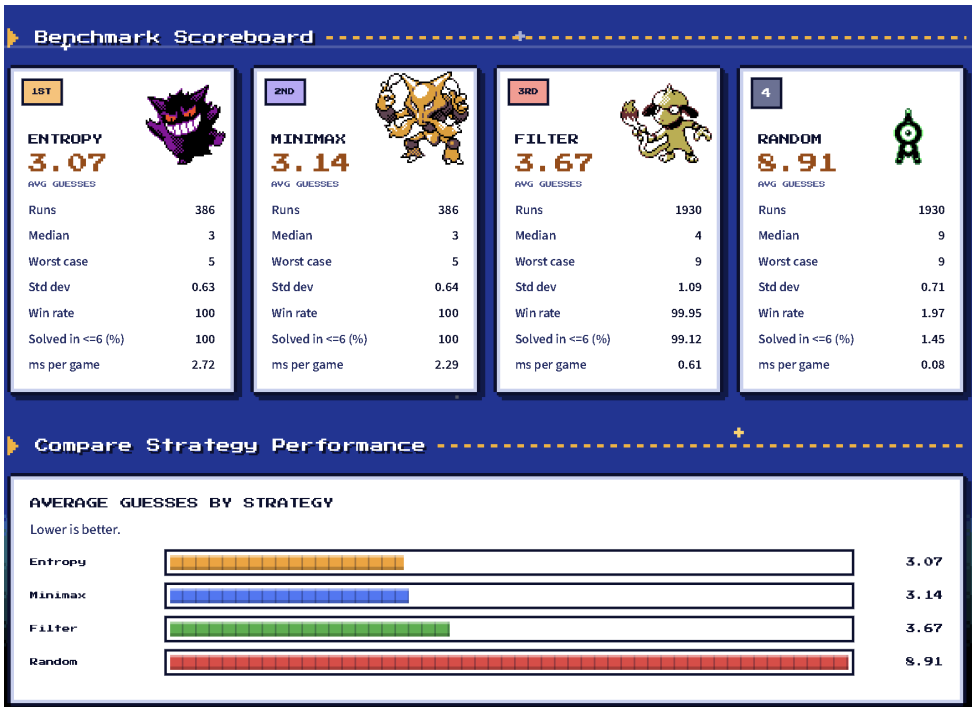


Figure 3: Algorithm Comparison: benchmark scoreboard and average guesses by strategy.

6. Experimental Setup

All four strategies were evaluated on the same dataset of 386 Pokémon records, with a maximum of eight guesses per game. The number of runs differed between strategies:

- **Random:** five runs for each of the 386 possible hidden Pokémon.
- **Filter:** five runs for each of the 386 possible hidden Pokémon.
- **Entropy:** one run for each of the 386 possible hidden Pokémon.
- **Minimax:** one run for each of the 386 possible hidden Pokémon.

Random and Filter therefore each completed 1,930 runs, while Entropy and Minimax each completed 386 runs. Random and Filter are non-deterministic and were repeated; Entropy and Minimax were run once per hidden Pokémon.

The following measures were recorded:

- average number of guesses per game,
- median number of guesses,
- worst-case score,
- standard deviation,
- win rate,
- percentage of games solved within six guesses, and
- average execution time per game in milliseconds.

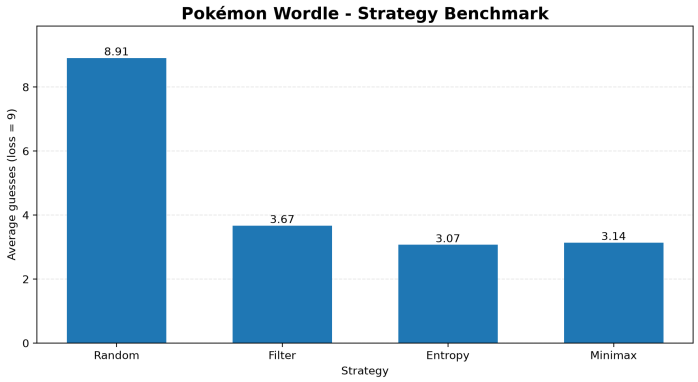


Figure 4: Comparison of average guesses by strategy (a lost game is scored as 9).

A successful game is scored by the number of guesses it required. An unsuccessful game receives a score of 9, following the project's `FAIL_SCORE` convention. The actual game limit remains eight guesses.

7. Results and Performance Comparison

7.1 Benchmark Results

Table 2 summarizes the benchmark results for the four strategies.

Table 2: Performance comparison of the guessing strategies

Strategy	Runs	Avg.	Median	Worst	Std. Dev.	Win (%)	≤ 6 (%)	ms/game
Random	1930	8.91	9	9	0.71	1.97	1.45	0.08
Filter	1930	3.67	4	9	1.09	99.95	99.12	0.61
Entropy	386	3.07	3	5	0.63	100.00	100.00	2.72
Minimax	386	3.14	3	5	0.64	100.00	100.00	2.29

Avg. = average guesses. A lost game receives a score of 9 (`FAIL_SCORE`); the actual game limit is eight guesses.

7.2 Benchmark Visualization

Figure 4 compares the average number of guesses for the four strategies.

7.3 Interpretation of Results

The benchmark results show clear differences in the efficiency of the four strategies.

- **Random:** averaged 8.91 guesses and won only 1.97% of games, which reflects its lack of feedback-based decision-making.
- **Filter:** averaged 3.67 guesses and won 99.95% of games. It solved 99.12% of games within six guesses.
- **Entropy:** achieved the lowest average, 3.07 guesses, and solved all 386 evaluated cases with a worst case of five guesses.
- **Minimax:** averaged 3.14 guesses and also solved all 386 evaluated cases with a worst case

of five guesses.

Entropy achieved the lowest average number of guesses, and Minimax produced comparable results. Filter also performed strongly while using less computation time than the two advanced strategies.

8. Discussion

The results highlight a trade-off between guessing efficiency and computational cost.

Random guessing performed poorly because it ignores the feedback from earlier guesses. Filter improved performance substantially by eliminating candidates that were inconsistent with the observed clues. Entropy and Minimax performed best, and both solved every evaluated case within five guesses.

Filter required less time per game (0.61 ms) than Entropy (2.72 ms) and Minimax (2.29 ms). Candidate elimination therefore offers a good balance between computational cost and guessing efficiency.

The strategies were evaluated with different numbers of runs. Random and Filter were run five times for each of the 386 hidden Pokémon (1,930 runs each), whereas Entropy and Minimax were run once for each hidden Pokémon (386 runs each). All games used a maximum of eight guesses, and unsuccessful games were given a score of nine under the `FAIL_SCORE` convention.

9. Conclusion

Pokémon Wordle demonstrates how different algorithmic approaches can be applied to an attribute-based guessing game. The project brings together an interactive Streamlit application, a modular game implementation, automated solvers, and a benchmarking framework.

Among the evaluated strategies, Entropy achieved the lowest average number of guesses (3.07). Minimax achieved comparable performance, with an average of 3.14 guesses and a worst case of five guesses. Filter offered an effective candidate-elimination approach at low computational cost, while Random served as the baseline for comparison.

Overall, the project illustrates the practical value of candidate elimination, information theory, and worst-case optimization in intelligent decision-making.

References

[1] Streamlit Documentation. https://docs.streamlit.io/

[2] Python Documentation. https://docs.python.org/3/

[3] pandas Documentation. https://pandas.pydata.org/docs/

[4] scikit-learn Documentation. https://scikit-learn.org/stable/

[5] Shannon, C. E. (1948). A Mathematical Theory of Communication. *Bell System Technical Journal*, 27(3), 379–423.